

Lat Language Charter

Project: Latticra Programming Language

Short name: Lat

Status: Design Draft v0.1

Purpose: Establish the permanent design direction for Lat before implementation begins.

1. Core Position

Lat is the native application, server, simulation, and tooling language for the Latticra ecosystem.

Lat is designed to occupy a deliberate sweet spot:

```
C speed and machine awareness
C++ power and abstraction
Python/Ruby readability and joy
Go-like approachability where it helps
without adopting Go's runtime assumptions
without adopting Rust's complexity burden
```

Lat should be beautiful and elegant first, but not decorative. Beauty in Lat means clarity, symmetry, teachability, correctness, and expressive power without chaos.

The founding sentence:

Lat is a beautiful native language for clear, fast, safe, and powerful software.

The Latticra stack should be:

```
C      - machine boundary, ABI, hardware-level foundation
C++    - system runtime, core infrastructure, high-performance internals
Lat    - applications, servers, simulations, tools, extensions, user-facing
software
```

Lat should not be a toy language. It should be friendly enough to learn first and strong enough to use professionally.

2. Design Mantra

Lat should be governed by four short principles:

```
Beautiful before clever.  
Fast by design.  
Safe by default.  
Power when requested.
```

A Lat program should read like a clear explanation of itself.

A Lat system should run close to the metal when required.

A Lat beginner should not be forced to understand every compiler theory concept before writing useful code.

A Lat expert should not feel trapped inside a slow or overprotected runtime.

3. What Lat Is Not

Lat is not intended to become:

```
Rust with softer syntax.  
Go with more features.  
Python with a compiler bolted on.  
C++ with a safety slogan.  
Ruby with static types added later.  
A research language that is beautiful only on paper.
```

Lat should learn from these languages without becoming a clone of any of them.

Lat should especially avoid two traps:

1. **The low-level pain trap** — where everything is powerful but difficult.
2. **The high-level softness trap** — where everything is pleasant but performance, correctness, and control are compromised.

Lat exists to stay between those traps.

4. The Performance Covenant

Lat must not sacrifice native performance as a default assumption.

Lat should compile ahead-of-time to efficient native code. A Lat program should be capable of C/C++-class performance when written with performance in mind.

The language should support:

```
value types
stack allocation
predictable layout
zero-cost abstractions where practical
monomorphized generics where appropriate
bounds-check elimination when proven safe
explicit heap allocation
explicit ownership transfer
explicit mutation
arena allocation
FFI with C and C++ boundaries
SIMD-friendly math types
native async/runtime options without mandatory global runtime tax
```

Lat should not require a mandatory tracing garbage collector for normal programs.

Lat may support managed memory tools, reference counting, arenas, and safe ownership patterns, but memory strategy should be explicit and appropriate to the job.

The rule:

Lat should make fast code natural, and slow code visible.

5. The Elegance Covenant

Lat should feel beautiful to write.

Beauty means:

```
minimal punctuation noise
consistent syntax
clear names
obvious control flow
low ceremony for common tasks
strong structure for serious systems
```

Lat should feel closer to this:

```
fn area(radius: F64) -> F64 {  
    return pi * radius * radius  
}
```

than this:

```
ceremony-heavy generic trait-bound lifetime-oriented boilerplate
```

Lat should not hide complexity by pretending it does not exist. It should reveal complexity gradually.

The beginner path should be smooth. The expert path should be deep.

6. The Safety Covenant

Lat should be safe by default, but not anti-machine.

Safe Lat should guarantee:

```
no null dereference by default  
no undefined behavior in safe code  
no unchecked buffer access  
no silent integer overflow by default  
no data races in safe concurrent code  
no accidental use-after-free  
no accidental double-free  
no ambient authority for filesystem or network access
```

Dangerous power must exist, but it must be marked.

```
unsafe "calling verified C graphics ABI" {  
    driver_submit_frame(raw_frame)  
}
```

Unsafe blocks should require a reason string. This creates an audit trail and teaches discipline.

The rule:

Power is allowed. Accidental danger is not.

7. The Simplicity Covenant

Lat should not make beginners fight the compiler.

Lat should use static typing, but with local type inference.

```
let name = "Ada"
let age = 37
let active = true
```

Public APIs should prefer explicit types:

```
pub fn distance(a: Point, b: Point) -> F64 {
    let dx = a.x - b.x
    let dy = a.y - b.y
    return sqrt(dx * dx + dy * dy)
}
```

Lat should not require advanced ownership annotations for ordinary beginner programs.

The beginner should learn this first:

```
let means stable value.
var means changeable value.
fn means function.
type means named structure.
match means handle all cases.
Result means something can fail.
Maybe means something can be absent.
```

Only later should they learn arenas, FFI, unsafe, async runtimes, and low-level layout control.

8. Memory Model Direction

Lat should use a gentle ownership model, not a hostile one.

The model:

```
read by default
mutate only when visible
consume only when explicit
```

```
allocate intentionally  
free automatically when ownership ends
```

Parameter modes:

```
fn inspect(value: T)           // read-only view by default  
fn change(mut value: T)       // exclusive mutable access  
fn consume(own value: T)      // ownership transfer
```

Call site visibility:

```
sort(mut numbers)  
let bytes = into_bytes(own text)
```

This makes important behavior readable without making all code feel like a proof exercise.

The design goal:

Lat should offer memory safety with less psychological weight than Rust and more discipline than C++.

9. Core Syntax Direction

Lat uses a clean C-family surface with modern restraint.

```
package examples.hello  
  
fn main(app: App) -> Exit {  
    app.out.println("Hello, Lat!")  
    return Exit.ok  
}
```

Basic values:

```
let x = 10  
var count = 0  
count += 1
```

Functions:

```
fn square(x: Int) -> Int {  
    return x * x  
}
```

Expression function form:

```
fn square(x: Int) -> Int => x * x
```

Records:

```
type Point = record {  
    x: F64  
    y: F64  
}
```

Choice types:

```
type Maybe[T] = choice {  
    Some(value: T)  
    None  
}
```

Pattern matching:

```
match user_result {  
    Ok(user) => app.out.println("Welcome {user.name}")  
    Err(error) => app.err.println("Could not load user: {error}")  
}
```

This style keeps Lat familiar without becoming boring.

10. Type System Direction

Lat should be statically typed with inference.

The type system should be strong enough for correctness, but not so abstract that ordinary application programmers feel punished.

Required features:

```
primitive numeric types
records
choice types
traits/interfaces
generics
Maybe for absence
Result for recoverable errors
arrays, lists, maps, sets
explicit unsafe escape hatches
```

Later features:

```
units of measure
matrix/vector shape types
contracts
effects/capability checking
static reflection or hygienic macros
```

Lat should not begin with the full advanced type system. It should earn complexity through real use.

11. Error Model

Lat should not use unchecked exceptions as the ordinary error model.

Recoverable errors use `Result`.

```
fn load_config(fs: Fs.Read, path: Path) -> Result[Config, ConfigError] {
  let text = fs.read_text(path)?
  return Config.parse(text)
}
```

Absence uses `Maybe`.

```
fn find_user(id: UserId) -> Maybe[User] {
  ...
}
```

Programmer errors may panic.

```
panic("impossible state reached")
```

The rule:

Expected failure is data. Unexpected failure is a bug.

12. Application and Capability Model

Lat should be designed for secure applications from the beginning.

Programs should not have ambient authority by default.

Bad direction:

```
any code can read files, open sockets, read env vars, and access the clock
```

Lat direction:

```
fn read_config(fs: Fs.Read, path: Path) -> Result[Config, Error] {  
    let text = fs.read_text(path)?  
    return Config.parse(text)  
}
```

A function can only use the powers it receives.

Application entry:

```
fn main(app: App) -> Result[Exit, Error] {  
    let config = read_config(app.fs.read, Path("./config.lat"))?  
    return run(config)  
}
```

This gives Latticra a strong native security model.

13. Math, Simulation, and Server Direction

Lat should not be only an application scripting language.

Lat should be designed for serious domains:

```
mathematics
simulation
servers
secure tools
native applications
plugins/extensions
system diagnostics
```

Math direction:

```
let velocity = distance / time
let next = position + velocity * dt
```

Simulation direction:

```
for step in 0..steps {
  world.advance(dt)
}
```

Server direction:

```
server.get("/health", fn(req: Request) -> Response {
  return Response.text("ok")
})
```

Lat should support numerical correctness, reproducibility, deterministic simulation, typed APIs, safe concurrency, and clear deployment.

14. Concurrency Direction

Lat should avoid casual shared-state concurrency.

Preferred tools:

```
structured tasks
async/await
channels
actors
```

isolated mutable state
capability-limited services

Example direction:

```
taskgroup group {  
  group.spawn(fn {  
    run_worker(1)  
  })  
  
  group.spawn(fn {  
    run_worker(2)  
  })  
}
```

The rule:

Concurrent work should have a visible lifetime.

No hidden orphan tasks by default.

15. First Programming Language Requirement

Lat should be teachable as a first programming language.

That means the official manual must begin with:

```
What is a program?  
What is a value?  
What is a type?  
What is a function?  
What is state?  
What is input and output?  
What is failure?  
What is abstraction?
```

Lat should not begin by teaching package management, async runtimes, lifetimes, macros, or compiler internals.

The first Lat learner should be able to write:

```
fn main(app: App) -> Exit {  
  app.out.println("I am learning Lat.")  
  return Exit.ok  
}
```

Then grow naturally into:

```
fn average(numbers: List[F64]) -> Maybe[F64] {  
  if numbers.len() == 0 {  
    return None  
  }  
  
  var total = 0.0  
  
  for number in numbers {  
    total += number  
  }  
  
  return Some(total / numbers.len().to_f64())  
}
```

16. Non-Goals for v0.1

Lat v0.1 should not attempt to include everything.

Do not include in v0.1 unless absolutely necessary:

```
full macro system  
advanced dependent typing  
complete async runtime  
graphical UI framework  
full server framework  
complex package registry  
large standard library  
multiple backends  
compiler plugins  
proof assistant features
```

Lat v0.1 should focus on the beautiful core.

17. v0.1 Required Scope

Lat v0.1 should include:

```
lexer
parser
formatter
clear compiler diagnostics
let / var
functions
basic types
records
choice types
match
Maybe
Result
for / while / loop
basic List and Map
modules/packages at a minimal level
tests
native compilation path
```

This is enough to prove Lat is real.

This is enough to begin The Lat Book.

This is enough to write small tools and examples.

18. v0.2 Direction

Lat v0.2 should add:

```
gentle ownership checking
mut and own parameter modes
safe heap allocation
basic capabilities
filesystem APIs
better package model
standard library growth
documentation generator
property tests
```

19. v0.3 Direction

Lat v0.3 should add:

```
C ABI interop
C++ boundary strategy
arenas
async foundations
channels
actors
server primitives
simulation/math package foundations
units of measure prototype
```

20. v1.0 Direction

Lat v1.0 should be ready when it can support:

```
real Latticra applications
real command-line tools
real server services
real math/simulation packages
safe FFI boundaries
capability-aware package manifests
stable formatter
stable core syntax
stable teaching manual
```

Lat v1.0 should not mean perfection. It should mean trust.

21. Cultural Direction

Lat should have a serious but welcoming culture.

Lat culture should value:

```
clarity
craft
performance
security
```

```
teaching
mathematics
systems thinking
beauty
kindness
rigor
```

Lat culture should reject:

```
needless complexity
elitist compiler worship
anti-beginner hostility
sloppy dynamic chaos
unsafe bravado
framework addiction
corporate blandness
hype without evidence
```

Lat should feel like a workshop, not a maze.

22. Final Direction Statement

Lat should move forward as:

A beautiful, native, statically typed language for Latticra applications, servers, simulations, and tools. It should preserve the speed and control spirit of C and C++, while offering the readability and joy of Python and Ruby, and the straightforward practicality people often seek from Go. Lat should be safer than C++, simpler than Rust, more powerful than Go, faster and more structured than Python or Ruby, and more coherent than a language assembled from trends.

The correct direction is not to chase every language.

The correct direction is to create the missing sweet spot.

```
Beautiful.
Native.
Safe.
Fast.
Clear.
Serious.
Fun.
```

That is Lat.